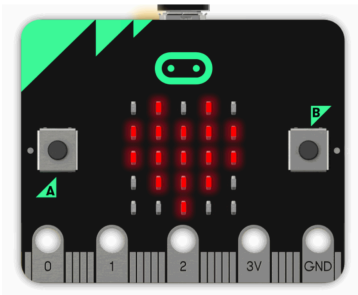
Lab 1: Programming in C and Assembly



Hilary Term 2026

Mark van der Wilk — Lecturer & Adaptations
Mike Spivey — Lecturer & Course Designer

Contents

1. Introduction 1
2. Building an Existing Program 2
3. Your Own Implementation 3
4. Tasks 3

ⓘ **Much to read, not as much to do. Read *before* the scheduled lab session!**

1. Introduction

This lab is built around a program, mostly written in C, that calls an assembly language subroutine to perform an arithmetic operation. As supplied, there are two versions of the subroutine – one that uses the machine's adds instruction to add two numbers, and another that contains a simple but slow loop that performs multiplication. Your task is to add other variations, such as subtraction (easy), a faster multiplication algorithm (moderate), or a fast division algorithm (tough).

The lab1-asm directory of the lab materials contains the following files, some of them the same as the corresponding files seen before:

File Name	Description
`Makefile`	Build script
`fmain.c`	Main program
`func.s`	Subroutine – single add instruction
`mul1.s`	Subroutine – simple multiplication loop
`fac.s`	Factorials with mult as a subroutine
`bank.s`	"Bank accounts" with a static array
`hardware.h`	Header file with layout of I/O registers
`lib.c, lib.h`	Library with implementation of printf
`startup.c`	Startup code
`nRF51822.ld`	Linker script
`debug`	Shell script for starting debugger
`qmain.c`	Alternative main program for use with QEMU

In particular, `main.c` is the main program, written in C, with a loop that prompts for two unsigned integers `x` and `y`, then calls a function `func(x, y)` and prints the result. This function is not specified in the `main.c` file, and therefore must be specified in a different one. The object code for `main.c` and whatever file `func()` is defined in, must be linked together before creating the final runnable program.

There's no reason why this function couldn't be written in C, but instead two versions written in assembly language are provided, as a starting point for experiments in programming at the machine level.

2. Building an Existing Program

As discussed in lab 0, to build an initial program, just use the command

```
$ make
```

and usual (or use Build>Make within Geany. As a default, the Makefile will use the subroutine from `func.s`, which consists of only one instruction to add its two arguments. The linker finally produces a binary file `func.hex`, which you can load into the micro:bit by dragging and dropping, or by using the command

```
$ cp func.hex /media/mike/MICROBIT
```

Now start `minicom`, reset the micro:bit, and expect an interaction like this one:

```
Hello micro:world!
Gimme a number: 3
Gimme a number: 4
func(3, 4) = 7
func(0x3, 0x4) = 0x7
```

Note that positive and negative decimal numbers are allowed as input, and also numbers written in hexadecimal with a leading `0x`. The result returned by `func(x, y)` is interpreted both as a signed number printed in decimal, and as an unsigned number printed in hexadecimal. You are welcome to modify the main program if you wish to change this behaviour.

A second implementation of the function `func(x, y)` is provided in the file `mul1.s`, which computes the product of `x` and `y` using a simple loop. You can build a program containing this definition of `func` with the command

```
$ make mul1.hex
```

The Makefile will interpret the argument and look for the file `mul1.s` to link together with the other code, instead of using `func.s`. This Makefile will be link any assembly file you specify in this way.

You can load this new hex file to the microbit with

```
$ cp mul1.hex /media/mike/MICROBIT
```

or a similar command. (Within Geany, with the `mul1.s` source file open, choose Build>Make me to build the program, and Build>Upload me to upload it.)

The subroutine works well for simple examples like `2 * 3 = 6`, but you will find that one of `10000000 * 2` and `2 * 10000000` is much slower than the other. You may also notice that the main program lights one of the LEDs on the micro:bit before calling `func()`, and switches it off again when `func()` returns, so that you can see how much time the subroutine is taking. We can use the LED signal together with an oscilloscope to get a precise timing for the subroutine, and you can try this in the lab if you like.

Two additional implementations of a subroutine called `func` are provided, drawing from examples in the lectures.

- In `fac.s`, the subroutine `func(x, y)` returns the factorial of `x`, ignoring `y`. The subroutine calls another subroutine `mult` to do the multiplications needed to calculate the factorial.
- In `bank.s`, a static array of 10 integers is allocated, much as if it were declared in C with

  ```
  static int account[10];
  ```

  The subroutine `func(x, y)` increments `account[x]` by `y` and returns its new value; the values in the array are remembered from one invocation of `func` to the next, as is normal for a static array.

3. Your Own Implementation

You can make your own implementation of the function func as follows:

- Copy an existing implementation to get the structure straight:

  ```
  $ cp func.s sub.s
  ```
- Edit the copy `sub.s` to replace the `adds` instruction with something appropriate to your wishes – perhaps a `subs` instruction.
- Use the following commands to assemble your code and link it into a binary file, and load the result into the micro:bit:

  ```
  $ make sub.hex
  $ cp sub.hex /media/mike/MICROBIT
  ```

In writing your own versions of `func`, you should note the calling conventions of the ARM chip. Failing to obey them may make the program go haywire. In particular:

1. The arguments `x` and `y` arrive in registers `r0` and `r1`.
2. When the subroutine returns, whatever is left in register `r0` is taken as the result.
3. The subroutine may modify the contents of registers `r0` and `r1`, and also the contents of `r2` and `r3`.
4. Unless it takes special steps to preserve their values, the subroutine should not modify the contents of other registers such as `r4` to `r7`. It may be that the main program is keeping nothing special in those registers, in which case trashing them will do no harm, but that's not something we should rely on.
5. The subroutine body should not mess with the stack pointer `sp`, or the link register `lr`. The link register contains the code address to which the subroutine will return via the final instruction `bx lr`, and overwriting this will result in chaos.

4. Tasks

Different participants in the course will have different amounts of experience with low-level programming, so you should choose whichever of the following tasks you find both possible and illuminating. Each requires you to produce an assembly language file containing a definition of the function `func`. For the first few, you can work most easily by tinkering with the supplied file `func.s`, but for later tasks you may like to make a new file to preserve your work.

> ⓘ **Requirements for S and S+**
>
> For an S grade, you must show a successfully running multi-line assembly function. For an S+ grade, use the debugger to step through your code, and demonstrate the change in state of the registers for assembly instructions in your assembly function. Alternatively, connect an oscilloscope, or demonstrate running the code through the `qemu-arm` emulator, as per the lab sheets in `/old`.

1. Try replacing the `adds` instruction with a different instruction, such and the `subs` instruction that subtracts instead of adding. Explain to yourself why a subtraction such as `2 - 5` seems to give a large positive result when it is interpreted as an unsigned number (we will discuss this in lectures at some point).

2. Explore other ALU operations such as the bitwise logical instructions `ands`, `orrs` and `eors`, or the shifts and rotates `lsls`, `lsrs`, `asrs` and `rors`. Use shifts and adds to write a function that multiplies one of its arguments by a small constant, such as 10.
3. Write a function that multiplies two numbers using the built-in multiply instruction `muls`, and compare it for speed with the supplied code. Note that the Nordic chip includes the optional single-cycle multiplier, but other instances of Cortex-M0 may have a slower multiplier or (I believe) none at all.
4. Write a faster software implementation of multiplication, using a log-time algorithm.
5. Write a simple implementation of unsigned integer division.
6. Write an implementation of unsigned division that runs in a reasonable time.

Some of these tasks we will look at in the lectures, others may be mentioned on a problem sheet. I don't think the overlap matters much, because different settings are good for focussing on different things. You can fruitfully discuss with your tutor the range of algorithms you might use for one of the tasks, but it's a waste of tutorial time to discuss details of assembly level syntax, when the compiler can tell you when you do something wrong in a split second!

7. Factorials are often used as an example of recursion, though a bad one because they can be computed with a simple loop. Rewrite the factorial program to use recursion and see how much worse it gets.
8. Factorials provide one way of computing binomial coeffcients since $\binom{n}{r} = \frac{n!}{r!(n-r)!}$; others include filling in Pascal's triangle row by row, or using a recurrence such as

$$\binom{n}{r} = \frac{n}{r * \binom{n-1}{r-1}}$$

or

$$\binom{n}{r} = \frac{n-r+1}{r} * \binom{n}{r-1}$$

with suitable boundary conditions. Implement one or more of these methods, perhaps using the unsigned division routine you implemented earlier. Which method is the best, in terms of both speed and freedom from overflow?

💪 **Practice makes perfect!**

These exercises are all great to help you practice assembly coding, which is a skill that is commonly tested in the exam. When revising, you may want to do these exercises, and verify your code by comparing the resulting output, as run on the micro:bit, against a calculator.